

8086 MUL & DIV Register Cheat Sheet

Multiplication & Division Operations

Instruction	Size	Implicit Setup (Load this first)	Source Operand (Command you write)	Result: Quotient / Lower	Result: Remainder / Upper
MUL	8-bit (Byte)	AL	8-bit Reg/Mem (e.g., MUL BL)	AX (Full 16-bit Product)	None (Fits in AX)
MUL	16-bit (Word)	AX	16-bit Reg/Mem (e.g., MUL CX)	AX (Lower 16 bits)	DX (Upper 16 bits)
DIV	8-bit (Byte)	AX (16-bit Dividend)	8-bit Reg/Mem (e.g., DIV CL)	AL (Quotient)	AH (Remainder)
DIV	16-bit (Word)	DX:AX (32-bit Dividend)	16-bit Reg/Mem (e.g., DIV BX)	AX (Quotient)	DX (Remainder)

Three Critical Rules to Remember

1. **No Constants Allowed:** You cannot write `MUL 5` or `DIV 10`. You must move the constant into a register first (e.g., `MOV BX, 5` → `MUL BX`).
2. **The "Clear DX" Trap (16-bit Division):** When doing a 16-bit DIV, the processor looks at both `DX` and `AX` together as one massive 32-bit number. If you only loaded your number into `AX`, you **must** write `MOV DX, 0` right before you divide, or Emu8086 will crash with a "Divide Overflow" error!
3. **Chaining Operations:** If you multiply a 16-bit number and immediately want to divide the answer (e.g., $A * B / C$), you **do not** clear `DX`. MUL automatically leaves its 32-bit answer sitting perfectly in `DX:AX`, which is exactly where DIV expects to find it.